UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	Adelia (2428240104)
Kelas	SI 5B
Judul project / topik API	Studio Foto – Paket Foto
Tautan repositori	https://github.com/adelia2428240104-coder/tugas1-restful-2428240104
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	22 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Sebelum mengambil screenshot Runner
Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:

API yang saya buat menggunakan topik Studio Foto dengan data utama berupa paket foto yang ditawarkan kepada pelanggan. API ini digunakan untuk menampilkan, menambahkan, mengubah, dan menghapus data paket foto, seperti nama paket, jenis foto, jumlah foto, durasi, dan harga. API dikembangkan menggunakan Node.js dan Express.js dengan metode RESTful API untuk mengelola request dan response dalam format JSON.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/photo-packages	Tidak	200	Menampilkan semua paket foto
2	GET	/photo-packages/:1	Tidak	200	Menampilkan paket foto berdasarkan ID
3	POST	/photo-packages	Ya	201	Menambahkan paket foto baru
4	PUT	/photo-packages/:{{mahasiswaId}}	Ya	200	Mengubah data paket foto berdasarkan ID
5	DELETE	/photo-packages/:{{mahasiswaId}}	Ya	204	Menghaspus paket foto berdasarkan ID
6	POST	/photo-packages/	Tidak	401	Menguji request tanpa API key
7	POST	/photo-packages/	Ya	400	Menguji request dengan body tidak lengkap
8	POST	/photo-packages/	Ya	400	Menguji request dengan format JSON tidak valid
9	GET	/photo-packages/99	Tidak	404	Menguji pencarian ID paket yang tidak ditemukan

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

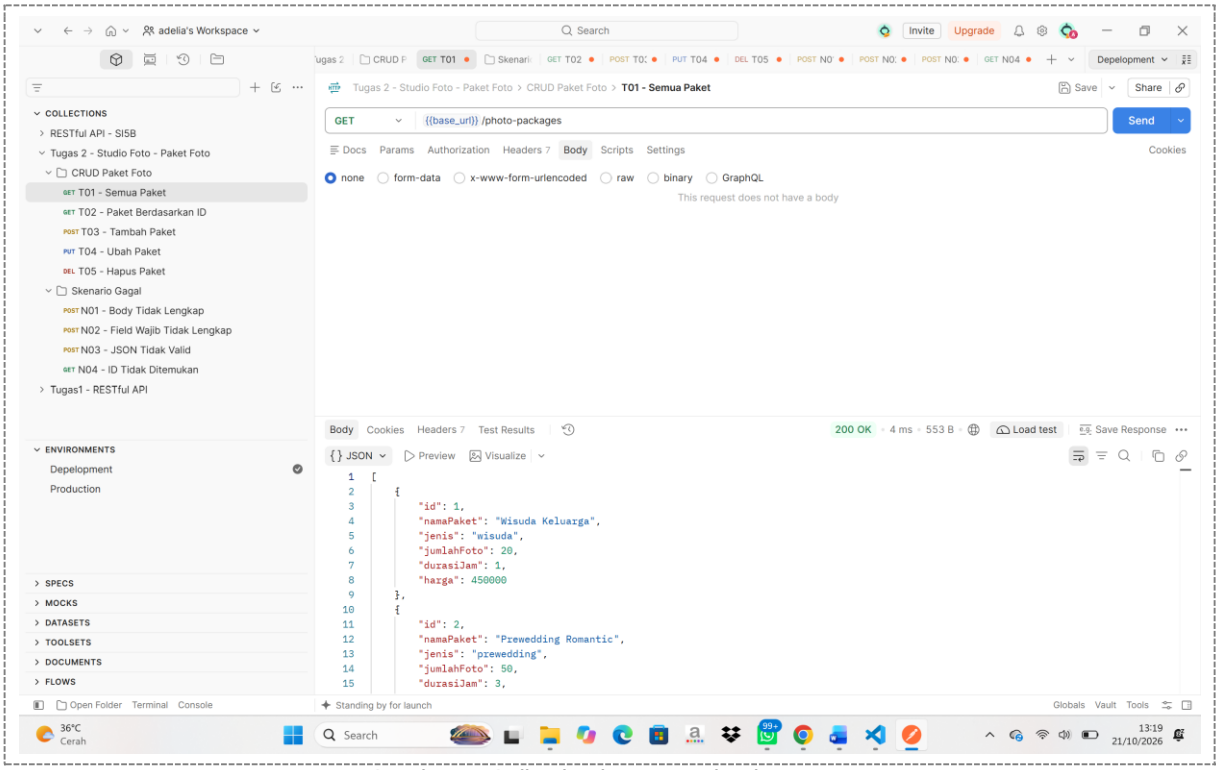
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Alamat server API yang akan diuji
2	api_key	Adelia2428240104	Tugas 1 belum menggunakan API key
3	mahasiswaid	(kosong, diisi script)	ID dari respons POST
4	Jenis	wisuda	Menyimpan jenis paket foto untuk pengujian filter

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:

Saya mengelompokkan request Postman ke dalam satu Collection bernama Tugas 2 - Studio Foto - Paket Foto. Request dibagi menjadi folder CRUD Paket Foto untuk pengujian proses menampilkan, menambah, mengubah, dan menghapus paket foto, serta folder Skenario Gagal untuk menguji respons API ketika menerima permintaan yang tidak sesuai. Pengelompokan ini memudahkan saya menemukan dan menjalankan setiap pengujian.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	T01 – Semua Paket	GET	{{base_url}}/photo-packages	CRUD Paket Foto
2	T02 – Paket Berdasarkan ID	GET	{{base_url}}/photo-packages/1	CRUD Paket Foto
3	T03 – Tambah Paket	POST	{{base_url}}/photo-packages	CRUD Paket Foto
4	T04 – Ubah Paket	PUT	{{base_url}}/photo-packages/{{mahasiswaId}}	CRUD Paket Foto
5	T05 – Hapus Paket	DELETE	{{base_url}}/photo-packages/{{mahasiswaId}}	CRUD Paket Foto
6	N01 – Body Tidak Lengkap	POST	{{base_url}}/photo-packages	Skenario Gagal
7	N02 – Field Wajib Tidak Lengkap	POST	{{base_url}}/photo-packages	Skenario Gagal
8	N03 – JSON Tidak Valid	POST	{{base_url}}/photo-packages	Skenario Gagal
9	N04 – ID Tidak Ditemukan	GET	{{base_url}}/photo-packages/99	Skenario Gagal
10	-	-	-	-

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	T01 – Semua Paket	pm.response.to.have.status(200)	pm.expect(pm.response.json()).to.be.an("array")
2	T02 – Paket Berdasarkan ID	pm.response.to.have.status(200)	pm.expect(pm.response.json()).to.have.property("id", 1)
3	T03 – Tambah Paket	pm.response.to.have.status(201)	pm.expect(pm.response.json()).to.have.property("id")
4	T04 – Ubah Paket	pm.response.to.have.status(200)	pm.expect(pm.response.json()).to.have.property("namaPaket")
5	T05 – Hapus Paket	pm.response.to.have.status(204)	pm.expect(pm.response.text()).to.eql("")
6	N01 – Body Tidak Lengkap	pm.response.to.have.status(401)	pm.expect(pm.response.json()).to.have.property("message")
7	N02 – Field Wajib Tidak Lengkap	pm.response.to.have.status(400)	pm.expect(pm.response.json()).to.have.property("message")
8	N03 – JSON Tidak Valid	pm.response.to.have.status(400)	pm.expect(pm.response.json().message).to.eql("Format JSON tidak valid")
9	N04 – ID Tidak Ditemukan	pm.response.to.have.status(404)	pm.expect(pm.response.json()).to.have.property("message")

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
pm.test("Status code harus 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Response memiliki data paket", function () {
    const jsonData = pm.response.json();

    pm.expect(jsonData).to.have.property("status", "success");
    pm.expect(jsonData).to.have.property("data");
    pm.expect(jsonData.data).to.be.an("array");
});

pm.test("Data paket memiliki atribut yang sesuai", function () {
    const jsonData = pm.response.json();
    const paket = jsonData.data[0];

    pm.expect(paket).to.have.property("id");
    pm.expect(paket).to.have.property("namaPaket");
    pm.expect(paket).to.have.property("jenis");
    pm.expect(paket).to.have.property("jumlahFoto");
    pm.expect(paket).to.have.property("durasiJam");
    pm.expect(paket).to.have.property("harga");
});
```

Penjelasan:

Pada request T01 – Semua Paket, saya membuat test script untuk memastikan API memberikan status code 200 ketika data paket foto berhasil diminta. Saya juga memeriksa bahwa response memiliki status success, terdapat atribut data berbentuk array, dan setiap paket foto memiliki atribut yang sesuai, seperti id, namaPaket, jenis, jumlahFoto, durasiJam, dan harga. Dengan pengujian ini, saya dapat memastikan bahwa API tidak hanya berhasil merespons request, tetapi juga mengembalikan data paket foto dengan struktur yang sesuai.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
pm.test("Status code harus 201", function () {
    pm.response.to.have.status(201);
});

pm.test("Response memiliki data paket yang ditambahkan", function () {
    const jsonData = pm.response.json();

    pm.expect(jsonData.status).to.eql("success");
    pm.expect(jsonData.data).to.have.property("id");
    pm.expect(jsonData.data).to.have.property("namaPaket");
});

pm.test("ID paket disimpan ke Environment", function () {
    const data = pm.response.json();

    pm.environment.set("mahasiswaId", data.data.id);
});
```

Penjelasan:

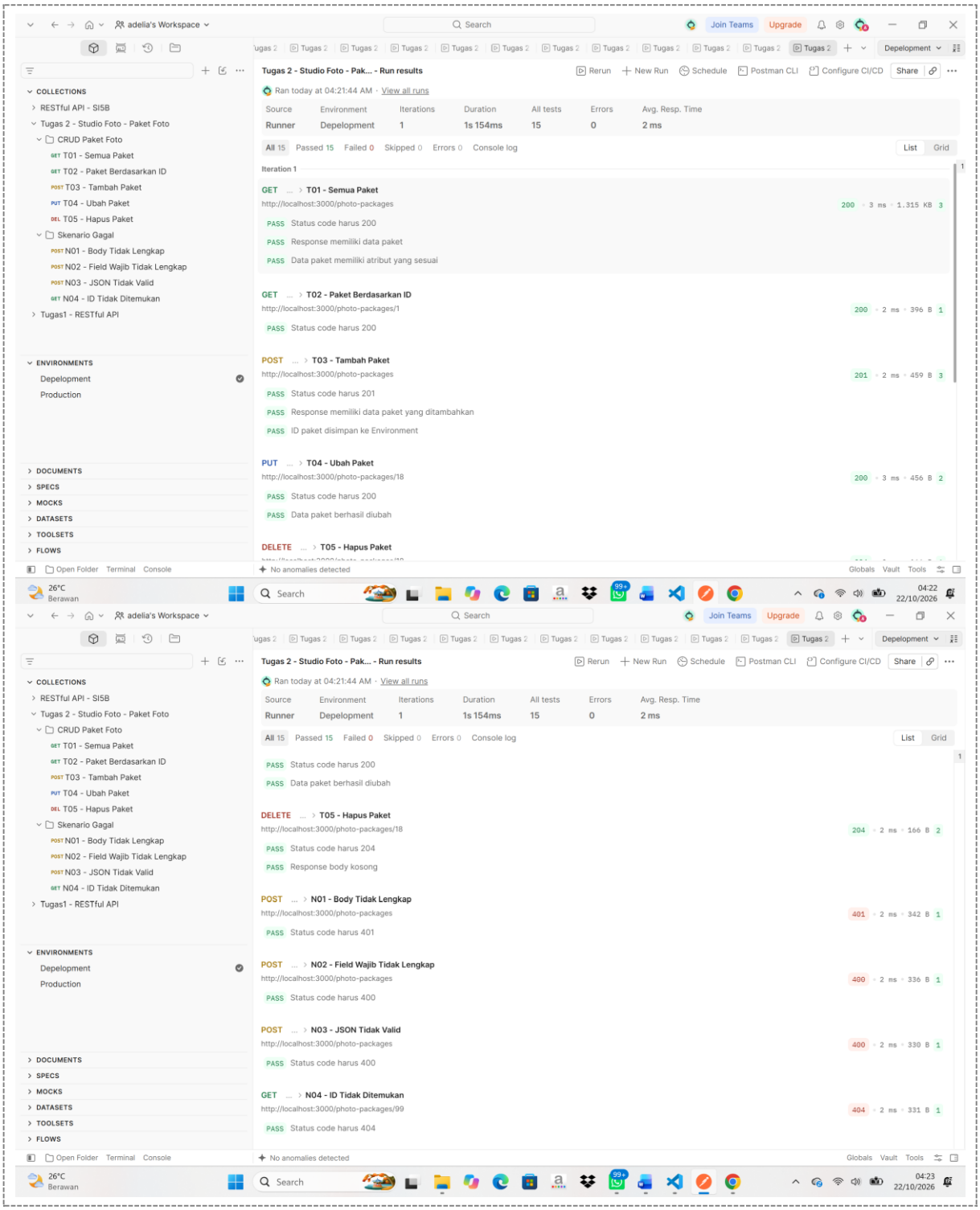
Pada request T03 – Tambah Paket, saya membuat test script untuk memastikan API memberikan status code 201 ketika paket foto berhasil ditambahkan. Saya memeriksa bahwa response memiliki status success serta data paket yang baru, termasuk atribut id dan namaPaket. Selain itu, ID paket yang diperoleh dari response disimpan ke Environment dengan nama variabel mahasiswaId, sehingga dapat digunakan kembali pada request PUT dan DELETE tanpa perlu memasukkan ID secara manual.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:

Berdasarkan hasil pengujian menggunakan Postman, terdapat 9 request utama yang diuji, dengan total 15 test yang dijalankan. Seluruh test berhasil lulus, yaitu 15 Passed, 0 Failed, dan 0 Errors. Pengujian meliputi operasi GET, POST, PUT, DELETE, serta empat skenario gagal untuk memeriksa penanganan kesalahan API.

Sebelum menjalankan Runner, saya memastikan server Express.js berjalan pada port 3000 dan API dapat diakses. Saya juga memeriksa konfigurasi request serta API key yang digunakan pada pengujian. Hasilnya, seluruh pengujian berjalan sesuai dengan status respons yang diharapkan.

4.2 Rekap angka hasil

Nama Collection	Tugas 2 – Studio Foto – Paket Foto
Environment yang dipakai	Development
Jumlah request dijalankan	9 request
Jumlah test lulus	15
Jumlah test gagal	0
Tanggal dan jam pengujian	22 Oktober 2026, 04.21 WIB

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	PUT /photo-packages/{{mahasiswaId}}	Response 404 Route tidak ditemukan	ID pada URL tidak sesuai dengan ID paket yang tersimpan atau digunakan oleh pengujian.	Menyesuaikan URL PUT menggunakan variabel {{mahasiswaId}} yang disimpan setelah POST berhasil.
2	DELETE /photo-packages/{{mahasiswaId}}	Response 404 Route tidak ditemukan	ID yang digunakan untuk menghapus paket tidak sesuai dengan ID paket yang tersedia.	Menggunakan ID dari hasil POST melalui {{mahasiswaId}} dan memastikan paket belum dihapus sebelum request DELETE dijalankan.

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.

[Tempel screenshot di sini]

Gambar 3. Hasil menjalankan Collection dengan Newman

Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Tulis perintah yang Anda jalankan dan jelaskan arti exit code (0 atau 1) pada hasil Anda.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/adelia2428240104-coder/tugas1-restful-2428240104
Lokasi berkas Collection (.json)	
Lokasi berkas Environment (.json)	
Berkas Environment tidak memuat kunci rahasia sungguhan	[ ] Ya [✓] Tidak
Repositori dapat dibuka dosen	[✓] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:

Assertion yang paling berguna menurut saya adalah assertion untuk memeriksa status code dan data response, karena dapat mengetahui apakah API memberikan hasil sesuai yang diharapkan. Saat pengujian, saya juga menemukan kesalahan pada request PUT dan DELETE ketika ID yang digunakan tidak sesuai, sehingga saya memperbaiki penggunaan variabel ID dari hasil request POST.

.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:

Menurut saya, pengujian manual lebih mudah untuk memeriksa satu request secara langsung, sedangkan Collection Runner lebih praktis karena dapat menjalankan banyak request beserta test secara berurutan. Bagian yang paling sulit adalah memastikan ID dari hasil POST digunakan dengan benar pada request PUT dan DELETE, sehingga saya mengatasinya dengan menyimpan ID ke environment dan memakainya kembali pada request berikutnya.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	Ya
2	Setiap endpoint API saya ada di Collection	Ya
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	Ya
4	Setiap request punya assertion status code dan struktur JSON	Ya
5	Nama setiap test berupa kalimat yang menyatakan harapan	Ya
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	Ya
7	Server dinyalakan ulang sebelum Runner dijalankan	Ya
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	Ya
9	Berkas Environment tidak memuat kunci rahasia sungguhan	Ya
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	Ya

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen